

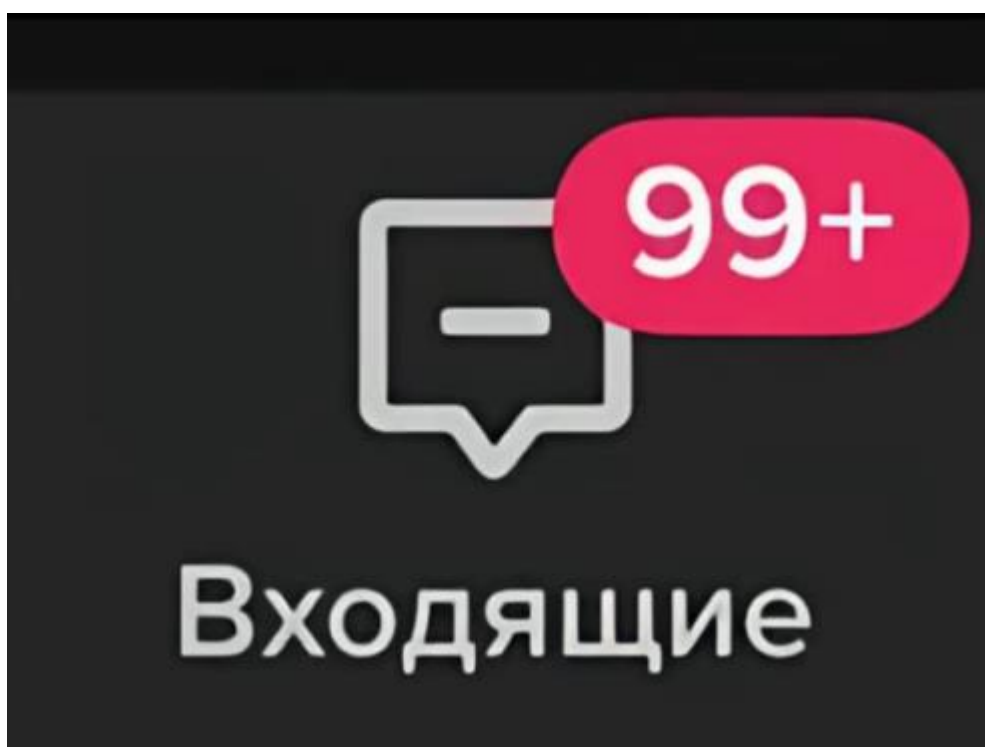
ЛАБОРАТОРНАЯ РАБОТА №9 ИСПОЛЬЗОВАНИЕ NOTIFICATION ДЛЯ СОЗДАНИЯ PUSH-УВЕДОМЛЕНИЙ Что делаем сегодня?

Сегодня мы попробуем **поработать с уведомлениями**. До этого все, что делала программа, происходило на экране. Уведомление живёт в системной шторке, то есть его видит система, а не ваша разметка.

Уведомление – непростая штука, за ним стоит несколько барьеров, придуманные **Google**, чтобы приложения не спамили ими.

Основные правила

1. Лабораторная работа разделена на **ЧАСТИ** – выполняете каждую из частей и **показываете**.
2. Обычно в конце каждой части есть **ЗАДАНИЕ** – его надо сделать
3. Не бойтесь **задавать любые** вопросы
4. Если хотите сделать что-то **свое**, то **предлагайте** – будем думать
5. Советую читать текст, тут **написано** много полезного
- 6.



Часть 1. Тема и разметка

Тема определяет лишь определит какой будет текст у уведомления, а код у всех будет практически одинаковым – изменяться лишь строки (либо можете добавить что-то свое).

Создайте экран по **макету**:

Вертикальный *LinearLayout*

Название

Введите текст...

Показать уведомление

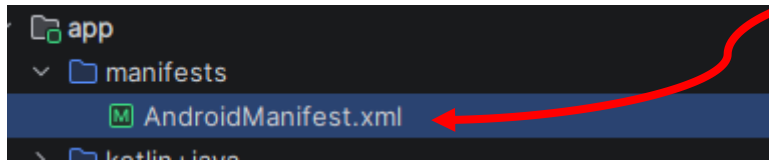
! ЗАДАНИЕ !

1. вынесите все строки из элементов в ресурсы **strings.xml**.
2. назначьте каждому элементу свой **id**, например:

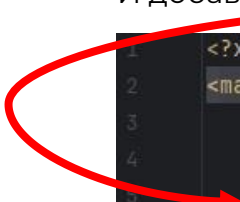
```
android:id="@+id/btnOrder"
```

Часть 2. Пробуем показать уведомления

Перед тем как работать с уведомлениями надо сказать системе, что мы вообще хотим пользоваться ими. Чтобы это сделать зайдем в **AndroidManifest.xml**:



И добавим в него эту **строчку**:



```
1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:tools="http://schemas.android.com/tools">
4
5     <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
6
7     <application
8         android:allowBackup="true"
9         android:dataExtractionRules="@xml/data_extraction_rules"
10        android:fullBackupContent="@xml/backup_rules"
11        android:icon="@mipmap/ic_launcher"
12        android:label="@string/NotificationLab"
13        android:roundIcon="@mipmap/ic_launcher_round"
14        android:supportsRtl="true"
15        android:theme="@style/Theme.NotificationLab">
16         <activity
17             android:name=".MainActivity"
18             android:exported="true"
19             android:windowSoftInputMode="adjustResize">
20             <intent-filter>
21                 <action android:name="android.intent.action.MAIN" />
22
23                 <category android:name="android.intent.category.LAUNCHER" />
24             </intent-filter>
25         </activity>
26     </application>
27 </manifest>
```

Теперь система знает, что наше приложение будет **POST_NOTIFICATIONS** (**СОЗДАВАТЬ_УВЕДОМЛЕНИЯ**).

Откроем **MainActivity** и создадим **канал** для нашего уведомления (не забываем про поля):

```
// задаем поля для уведомлений
private static final String CHANNEL_ID = "reminders"; 2 usages
private int notificationId = 1; 1 usage

private EditText etText; 2 usages

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    EdgeToEdge.enable(this);
    setContentView(R.layout.activity_main);
    ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (View v, WindowInsetsCompat insets) -> {
        Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
        v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
        return insets;
    });

    etText = findViewById(R.id.etText);
    createChannel();
}

private void createChannel() { 1 usage
    // создаем канал (это что-то по типу категории)
    // задаем ему айди, название и важность
    NotificationChannel channel = new NotificationChannel(
        CHANNEL_ID,
        name: "Напоминания",
        NotificationManager.IMPORTANCE_DEFAULT
    );
    // говорим системе - создай наш channel
    NotificationManager manager = getSystemService(NotificationManager.class);
    manager.createNotificationChannel(channel);
}
```

Важность канала бывает разная:

<u>Важность</u>	<u>Поведение</u>
IMPORTANCE_HIGH	более заметное уведомление
IMPORTANCE_DEFAULT	обычное уведомление
IMPORTANCE_LOW	менее заметное
IMPORTANCE_MIN	минимальная заметность

Теперь сделаем метод, который будет принимать текст и отображать его в уведомление:

```
@SuppressWarnings("MissingPermission") 1 usage
private void sendNotification(String text) {
    Notification notification = new NotificationCompat.Builder(context: this, CHANNEL_ID)
        .setSmallIcon(android.R.drawable.ic_dialog_info)
        .setContentTitle("Напоминание")
        .setContentText(text)
        .build();

    NotificationManagerCompat.from(context: this)
        .notify(notificationId++, notification);
}
```

Сделаем метод, который будет вызываться кнопкой для для проверки ввели ли мы текст и передачи его в уведомление:

```
private void showReminder() { no usages
    String text = editText.getText().toString().trim();

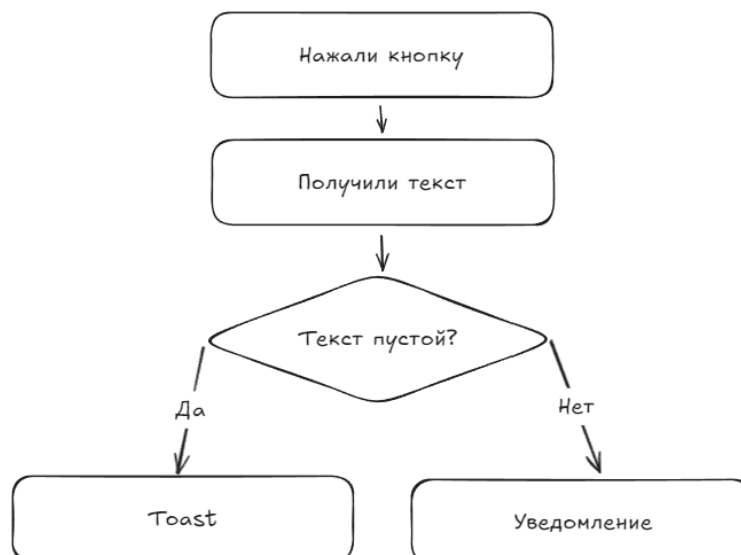
    if (text.isEmpty()) {
        Toast.makeText(context: this, text: "Введите текст", Toast.LENGTH_SHORT).show();
        return;
    }

    sendNotification(text);
}
```

Не забудьте **привязать** метод к нажатию кнопки:

```
findViewById(R.id.btnShow).setOnClickListener(View v -> showReminder());
}
```

Получается примерно такая **цепочка**:



! ЗАДАНИЕ !

Запустите и попробуйте показать уведомление. Появилось ли уведомление?

Часть 3. Разрешение на уведомления

Мы сказали системе в **AndroidManifest.xml**, что будем создавать уведомления, НО мы не спросили пользователя:

Разрешаешь ли приложению показывать уведомления?

Это стало необходимо делать начиная с Android 13. Потому давайте попросим пользователя нам разрешить отправку увед, допишем **метод showReminder**:

```
private void showReminder() { 1 usage
    // получаем текст, trim() удаляет лишние пробелы
    String text = editText.getText().toString().trim();

    // проверяем текст на пустоту
    if (text.isEmpty()) {
        Toast.makeText(context, this, text: "Введите текст", Toast.LENGTH_SHORT).show();
        return;
    }

    // проверяем -
    // если версия Android13+ && мы не можем отправлять уведомления
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU
        && checkSelfPermission(
            Manifest.permission.POST_NOTIFICATIONS
        ) != PackageManager.PERMISSION_GRANTED) {

        // запрашиваем разрешение на отправку уведомлений
        requestPermissions(
            new String[]{Manifest.permission.POST_NOTIFICATIONS},
            requestCode: 100
        );
        // оповещаем пользователя
        Toast.makeText(
            context: this,
            text: "Разрешите уведомления и нажмите кнопку ещё раз",
            Toast.LENGTH_SHORT
        ).show();

        return;
    }
    // иначе - отправляем уведомление
    sendNotification(text);
}
```

! ЗАДАНИЕ !

Запустите и попробуйте показать уведомление. Появилось ли теперь уведомление? Что будет если нажать на уведомление?

Часть 4. Сделаем уведомление нажимаемым

Сейчас по нажатию на уведомление ничего **не происходит**, давайте сделаем так чтобы по нажатию на него **открывался наш экран**.

Изменим метод, в котором мы создавали наше уведомление (**sendNotification**):

```
@SuppressWarnings("MissingPermission") 1 usage
private void sendNotification(String text) {
    // создаем НАМЕРЕНИЕ
    Intent intent = new Intent( packageContext: this, MainActivity.class);
    PendingIntent pendingIntent = PendingIntent.getActivity(
        context: this,
        requestCode: 0,
        intent,
        PendingIntent.FLAG_IMMUTABLE
    );

    Notification notification = new NotificationCompat.Builder( context: this, CHANNEL_ID)
        .setSmallIcon(android.R.drawable.ic_dialog_info)
        .setContentTitle("Напоминание")
        .setContentText(text)
        .setContentIntent(pendingIntent)    // помещаем наше НАМЕРЕНИЕ в уведомление
        .setAutoCancel(true)              // после нажатия должно закрыться
        .build();

    NotificationManagerCompat.from( context: this)
        .notify(notificationId++, notification);
}
```

Впервые появилось слово **Intent**:

1. **Intent** (Намерение) – это намерение (или же можно сказать запрос) совершить какое-то действие
2. **PendingIntent** (ОтложенноеНамерение) – это отложенный Intent, который вы передаете кому-то для выполнения от имени вашего приложения в будущем времени – не прямо сейчас.

! ЗАДАНИЕ !

Запустите и попробуйте показать уведомление. Что будет если нажать на уведомление?

Требования к итоговой версии

1. **Канал** создаётся при старте, а **уведомления** идут через него
2. **Разрешение** запрашивается в момент первой отправки
3. Тап по уведомлению **открывает** приложение и убирает уведомление
4. Строки – в **ресурсах**

Дополнительно (по желанию)

1. **Своя иконка**. Сделайте у уведомления свою иконку
2. **Длинный текст**. Сделайте так, чтоб уведомление можно было развернуть